DEMS

Digital Evidence Management System

[***GITHUB_LINK***]

Repository Manual & User Guide

FastAPI • PostgreSQL • React • Docker

Team Members

- Goti Achyut Yatinbhai – B240184CS
- Nihal Kumar – B240254CS
- Dhameliya Krish Vedantkumar – B240167CS
- Prashant Raj – B240272CS

1. Project Overview

This repository contains a Digital Evidence Management System (DEMS) implementing a law-enforcement style workflow for managing cases, evidence, and custody chains.

What this repository implements

- A FastAPI backend (dems_backend/app) for authentication, user administration, case lifecycle, evidence handling, custody chain, and audit logging.
- A PostgreSQL database (configured directly or via Docker Compose).
- A React frontend (dems_backend/dems_frontend) for role-based operations.
- Local evidence file storage on disk (evidences/), with metadata stored in PostgreSQL.

Problem Domain Workflow

1. Admin creates users (inspector / officer / admin).
2. Inspector creates a case and assigns officers.
3. Inspector or officer adds evidence to the case.
4. Inspector records custody transfer and possession history.
5. Every important operation writes to an audit trail.

---

2. Repository Structure

```
DBMS/
├── README.md
└── dems_backend/
    ├── app/
    │   ├── main.py          # FastAPI bootstrap, CORS, scheduler, routers
    │   ├── config.py        # Environment settings model
    │   ├── database.py      # SQLAlchemy engine/session/base
    │   ├── models.py        # ORM models / table definitions
    │   ├── oauth2.py        # JWT creation + verification
    │   ├── utils.py         # password hash/verify, audit helper, emails
    │   ├── routers/         # API routes grouped by feature
    │   │   ├── auth.py
    │   │   ├── users.py
    │   │   ├── cases.py
    │   │   ├── evidence.py
    │   │   ├── custody.py
```

```
    │   │   └── audit.py
    │   └── schemas/         # Pydantic request/response schemas
    ├── dems_frontend/
    │   ├── App.jsx          # Main UI (role-based views)
    │   ├── api.js           # Frontend API client wrapper
    │   ├── main.jsx
    │   ├── index.html
    │   ├── package.json
    │   └── vite.config.js
    ├── Dockerfile
    ├── docker-compose.yml
    ├── requirements.txt
    ├── .env.example
    └── .env
```

---

3. Technology Stack

Layer	Technology / Details
API / Backend	Python 3.13, FastAPI, SQLAlchemy 2.x, Sqlalchemy
Authentication	python-jose (JWT), passlib + bcrypt_sha256 (password hashing)
Scheduling	APScheduler (periodic background checks)
Frontend	React 18, Vite 5, plain JavaScript (no TypeScript in active entry files)
Database	PostgreSQL 18 (Docker image used in Compose)

---

4. Architecture and Runtime Flow

High-Level Architecture

```
[React Frontend]
      |
      | HTTP + Bearer JWT
      v
[FastAPI Backend]
      | \
      |  \---> local filesystem (evidences/case_xxx/...)
      v
[PostgreSQL]
```

App Startup Behavior (app/main.py)

- Creates scheduler and configures missing-file job.
- Registers startup and shutdown handlers for scheduler.

- Calls Base.metadata.create_all(bind=engine) to create tables if absent.
- Enables CORS for http://localhost:5173 and http://127.0.0.1:5173
- Includes all routers: users, cases, evidence, custody, audit, auth

---

5. Data Model

5.1 Tables and Columns

users

- UserID (PK)
- Name (required)
- Role (required; values: admin, inspector, officer)
- BadgeNumber (required, unique)
- Contact (nullable)
- Status (default: ACTIVE)
- LastLogin (nullable timestamp with tz)
- Password (required hash)
- Email (required)

cases

- CaseID (PK)
- Title (required)
- Type (required)
- Status (required)
- DateOpened (default now)
- DateClosed (nullable)
- Description (nullable text)
- ActingInspectorID (int, required)

case_assignments

- id (PK)
- CaseID (FK -> cases.CaseID, cascade on delete)
- AssignedOfficerId (FK -> users.UserID, cascade on delete)

evidenceitems

- EvidenceID (PK)
- CaseID (FK -> cases.CaseID, set null on delete)
- Description (nullable)
- EvidenceType (required)
- SourceOrigin (required)

- DateCollected (default now)
- SubmittingOfficerID (FK -> users.UserID, set null on delete)
- FilePath (nullable, path to physical file)

custodyrecords

- RecordID (PK)
- EvidenceID (FK -> evidenceitems.EvidenceID, set null on delete)
- Timestamp (default now)
- ActingOfficerID (FK -> users.UserID, set null on delete)
- Notes (nullable)

auditlog

- LogID (PK)
- Timestamp (default now)
- UserID (FK -> users.UserID, set null on delete)
- EventType (READ / CREATE / UPDATE / DELETE)
- Details (text)

5.2 Important Design Note

Important: Evidence files are stored on disk, not in DB blobs. Database backup alone is not enough — you must also back up the `evidences/` directory.

---

6. Authentication and Authorization

6.1 Auth Flow

6. Client posts to POST /login using form data (username = badge number, password).
7. Backend verifies hashed password via Passlib.
8. Backend checks user status is ACTIVE.
9. Backend issues JWT with payload { BadgeNumber, exp }.
10. Client calls protected endpoints using: Authorization: Bearer <token>

6.2 Current User Resolution

- oauth2.get_current_user decodes JWT, extracts BadgeNumber, fetches user from DB.
- Raises HTTP 401 if token is invalid or missing.

6.3 Role Model

- admin — user management, audit log access, delete operations
- inspector — case management, evidence handling, custody records
- officer — view and work assigned cases and evidence

Role checks are enforced server-side inside each router function.

---

7. Feature-by-Feature Behavior

7.1 Users Module (/users)

Main Capabilities

- Admin creates users; badge number and temporary password are auto-generated.
- Credentials are emailed to the user.
- Admin can list, update, and delete users.
- Any logged-in user can change their own password (/users/change-password).
- Admin/Inspector can fetch active officers (/users/officers/active).

Notable Implementation Details

- Create route expects: Name, Role, Contact, Email, Status.
- Password is NOT accepted from request body during user create/update.
- Password change requires the correct old password.

7.2 Cases Module (/cases)

Main Capabilities

- Inspector creates cases and can assign officers.
- Inspector can update, close, and manage officers on their own cases.
- Officer can view assigned cases only.
- Admin/Inspector can list all cases.
- Admin can delete cases.

Key Business Rules

- Only an inspector can create cases.
- Only the acting inspector can mutate assignments or case details.
- Close sets status to inactive and records DateClosed.
- Reactivate only works if case is currently inactive.

7.3 Evidence Module (/evidence)

Main Capabilities

- Inspector or assigned officer uploads evidence files.
- Any authenticated user can list and download evidence for a case.
- Admin can delete evidence and the underlying file.

File Handling Behavior

- Upload endpoint uses multipart/form-data.
- Server creates folder: evidences/case_<CaseID>/
- Saved filename pattern: case_id<CaseID>_evidence_id<EvidenceID>.<ext>

Integrity Monitoring

- utils.check_missing_files scans DB paths and emails alert if a file is missing.
- Scheduler in main.py runs this job periodically.

7.4 Custody Module (/custody)

Main Capabilities

- Inspector can add custody records.
- All authenticated users can read custody records.
- Inspector can update; admin can delete custody records.

Key Business Checks

- Evidence must exist; linked case must be active.
- Inspector must be acting inspector of that case to add custody.
- Duplicate (EvidenceID, ActingOfficerID) pair is blocked.

7.5 Audit Module (/audit)

- Admin-only endpoint to query audit logs.
- Supports filters: user_id, search, from_date, to_date, limit, skip.
- Most business operations write an audit log row via create_log.

---

8. API Endpoint Catalog

All protected routes require bearer token. No business routes are public.

Auth

- POST /login
- GET /me

Users (admin)

- POST /users/
- GET /users/

- GET /users/officers/active (admin, inspector)
- PUT /users/{badge_num}
- DELETE /users/{badge_num}
- POST /users/change-password (all authenticated)

Cases

- POST /cases/ (inspector)
- GET /cases/ (admin, inspector)
- GET /cases/assigned (inspector, officer)
- GET /cases/assigned/{officer_id} (admin, inspector)
- GET /cases/assigned-officers/{case_id}
- POST /cases/{case_id}/assign (inspector-owner)
- POST /cases/{case_id}/remove-officers (inspector-owner)
- PUT /cases/{case_id} (inspector-owner)
- PUT /cases/{case_id}/close
- PUT /cases/{case_id}/reactivate
- DELETE /cases/{case_id} (admin)

Evidence

- POST /evidence/ (inspector-owner, assigned officer)
- GET /evidence/case/{case_id} (all authenticated)
- GET /evidence/{case_id}/{evidence_id}/download
- PUT /evidence/{case_id}/{evidence_id}
- DELETE /evidence/{evidence_id} (admin)

Custody

- POST /custody/ (inspector-owner)
- GET /custody/ (all authenticated)
- GET /custody/{record_id}
- PUT /custody/{record_id} (inspector)
- DELETE /custody/{record_id} (admin)

Audit

- GET /audit/ (admin only)

---

9. Roles and Permissions

Capability	Admin	Inspector	Officer
Create / list / update / delete users	Yes	No	No
View audit log (GET /audit/)	Yes	No	No
Delete case / evidence / custody record	Yes	No	No
Create case; manage assignments; update/close own cases	No	Yes (own)	No
List all cases (with filters)	Yes	Yes	No
See "my" cases	No	Own as inspector	Assigned only
Add / list / download / update evidence	No	Own cases	Assigned cases only
Create custody record	No	Yes	No
Update custody record	Yes	Yes	No
List custody records	Yes	Yes	Yes

Inactive users (Status not ACTIVE) cannot authenticate, regardless of role.

---

10. Environment Configuration

Copy dems_backend/.env.example to .env inside dems_backend and set the following variables:

```
DATABASE_HOSTNAME=localhost
DATABASE_PORT=5433
DATABASE_PASSWORD=your_password
DATABASE_NAME=fastapi
DATABASE_USERNAME=your_name

SECRET_KEY=replace_with_a_long_random_secret
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

sender_mail=your_sender@gmail.com
superadmin_mail=your_admin@gmail.com
app_password_mail=your_gmail_app_password
app_scheduling_time=5
```

Email Notes

- Gmail SMTP (smtp.gmail.com:587) is used in utils.py.
- app_password_mail must be a Gmail App Password, not a regular account password.

---

11. Setup and Run Guide

Option A: Docker Compose (Recommended)

From dems_backend:

```
docker compose up --build
```

This starts the API on http://localhost:8000 and PostgreSQL on localhost:5433.

Option B: Run Backend Directly

From dems_backend:

```
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Interactive API docs: http://127.0.0.1:8000/docs (Swagger UI)

Health check: GET / returns {"status": "running"}

Run the Frontend

From dems_backend/dems_frontend:

```
npm install
npm run dev
```

The Vite dev server proxies all API routes to the backend. Open the URL Vite prints (typically http://localhost:5173).

---

12. First Admin Bootstrap

There is no built-in seed script. You need at least one row in users with Role = 'admin' and a bcrypt-hashed password before using full admin workflows.

Suggested Bootstrap Method

11. Open a Python shell in dems_backend.
12. Generate a password hash via app.utils.hash.
13. Insert an admin row into PostgreSQL with that hash.

After the initial admin exists: log in from the UI and create remaining users via /users/.

---

13. Operations, Backup, and Maintenance

Backup Strategy

You must back up both of the following together:

- PostgreSQL database
- evidences/ directory

Restoring only the DB will cause evidence file paths to break.

Missing File Monitor

- Scheduler periodically calls check_missing_files.
- If missing files are found, an email alert is sent to superadmin_mail.

Health Check

```
GET /  →  { "status": "running" }
```

---

14. Security and Governance Notes

Strengths Present in Code

- Password hashing (bcrypt_sha256 via Passlib).
- JWT-based authentication.
- Role checks enforced in backend routes.
- Audit logging on most critical actions.
- User status enforcement on login.

Risks and Improvement Points

- Base.metadata.create_all() is useful for dev but Alembic migrations are preferred for production.
- Secrets must never be committed in source control.
- Ownership checks should be tested end-to-end before deployment.
- Consider file storage hardening: checksums, malware scanning, immutable archive strategy.

15. Quick API Testing Snippets

Login

```
curl -X POST "http://localhost:8000/login" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=ADM12345&password=your_password"
```

Get Profile

```
curl "http://localhost:8000/me" \
  -H "Authorization: Bearer <ACCESS_TOKEN>"
```

List Cases

```
curl "http://localhost:8000/cases/?limit=20&skip=0" \
  -H "Authorization: Bearer <ACCESS_TOKEN>"
```

Upload Evidence

```
curl -X POST "http://localhost:8000/evidence/" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -F "CaseID=1" \
  -F "Description=Knife recovered near scene" \
  -F "EvidenceType=Weapon" \
  -F "SourceOrigin=Crime Scene" \
  -F "file=@/path/to/photo.jpg"
```

Change Password

```
curl -X POST "http://localhost:8000/users/change-password" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"oldPassword": "old_password_here", "newPassword": "new_password_here"}'
```

— End of DEMS Manual —